

Mars Operations: Base Deployment

Documentazione completa di sviluppo

Corso: *Metodi di Ottimizzazione per Big Data*

Prof. Andrea Cristofari

Università di Roma “Tor Vergata” — A.A. 2025/2026

Questo documento descrive in dettaglio la formulazione del problema MOBD, il fondamento teorico dell’algoritmo scelto (Randomized Block Coordinate Descent nella variante Gauss-Seidel con stepsize decrescente), le scelte progettuali adottate, i limiti incontrati e le soluzioni implementate.

Ogni blocco del codice `mobd_optimizer.py` è commentato riga per riga.

Indice

1	Descrizione del problema	2
1.1	Contesto e obiettivo	2
1.2	I quattro componenti di costo	2
2	Fondamento teorico dell’algoritmo	3
2.1	Ottimizzazione non vincolata e condizioni di ottimalità	3
2.2	Metodi a discesa: direzioni gradient-related	3
2.3	Block Coordinate Descent (BCD)	4
2.4	Randomized BCD (RBCD) — Gauss-Seidel con permutazione casuale	4
2.5	Stepsize decrescente (Diminishing Stepsize)	4
2.6	Differenze finite in avanti per il gradiente di f_4	5
3	Limiti iniziali e soluzioni adottate	6
3.1	Problema 1: costo computazionale di f_4	6
3.2	Problema 2: coerenza della base per le differenze finite	6
3.3	Problema 3: inizializzazione e trappole locali	6
3.4	Problema 4: I/O intensivo su disco	7
3.5	Problema 5: gradiente di f_1 di complessità $O(m^2)$	7
4	Struttura del codice: panoramica	7
5	Analisi riga per riga del codice	8
5.1	Import e costanti globali	8
5.2	Interfaccia con il simulatore	9
5.3	Funzioni di costo analitiche	12
5.4	Gradienti parziali analitici per blocco	13
5.5	Warm start	15
5.6	Loop principale RBCD	15
5.7	Modalità operative: fast, full, both	18
6	Convergenza: garanzie teoriche	19
6.1	Verifica delle condizioni per la convergenza	19
6.2	Complessità	20
6.3	Accuratezza del gradiente di f_4	20
7	Riepilogo delle scelte progettuali	20
8	Come eseguire il codice	20
9	Vincoli e regole della competizione rispettati	22
10	Conclusioni	22

1 Descrizione del problema

1.1 Contesto e obiettivo

Il progetto *Mars Operations: Base Deployment* (MOBD) simula il posizionamento ottimale di $m = 1000$ moduli operativi $x^1, \dots, x^m$ su una mappa bidimensionale del pianeta Marte. Nell'area è già presente una postazione fissa $z^0 = (0, 0)$ (centro di controllo) e quattro stazioni:

$$z^1 = (100, 100), \quad z^2 = (-100, 100), \quad z^3 = (-100, -100), \quad z^4 = (100, -100).$$

Ogni modulo $x^i \in \mathbb{R}^2$ ha coordinate (x_1^i, x_2^i) . Il vettore delle variabili di decisione è

$$x = (x_1^1, x_2^1, x_1^2, x_2^2, \dots, x_1^{1000}, x_2^{1000})^\top \in \mathbb{R}^{2000}.$$

Si vuole **minimizzare** la funzione di costo totale:

$$\min_{x \in \mathbb{R}^{2000}} f(x) = f_1(x) + f_2(x) + f_3(x) + f_4(x).$$

1.2 I quattro componenti di costo

f_1 — Dispersione della rete (coesione mesh)

$$f_1(x) = 0.1 \sum_{i < j} \left(1 - e^{-\|x^i - x^j\|^2} \right)$$

Misura la *perdita di coesione* della rete mesh. Per ogni coppia (i, j) , il termine $1 - e^{-\|x^i - x^j\|^2}$ è vicino a 0 quando i moduli sono vicini, e tende a 1 quando si allontanano. Si sommano tutte le $\binom{1000}{2} = 499\,500$ coppie.

f_2 — Distanza dalle stazioni fisse

$$f_2(x) = \frac{1}{m} \sum_{i=1}^m \sum_{q=0}^4 \|x^i - z^q\|^2$$

Penalità quadratica sull'allontanamento dalle $Q = 5$ stazioni fisse.

f_3 — Deviazione dalla distanza operativa ideale

$$f_3(x) = 1000 \sum_{i=1}^m \left(\frac{\log(1 + \|x^i - s^i\|^2)}{\log(1 + r^2)} - 1 \right)^2, \quad r = 100$$

dove $s^i = ((-1)^i \cdot i \cdot 0.2, (-1)^i \cdot i \cdot 0.2)$ è il punto di riferimento operativo del modulo i . La funzione è minimizzata (vale 0) esattamente quando $\|x^i - s^i\| = r = 100$ per ogni i .

f_4 — Costo ambientale black-box

f_4 modella fenomeni non ideali (ombreggiamento radio, micro-fratture del regolite, polvere, interferenze elettromagnetiche, congestione locale). **Non ha espressione analitica nota:** viene valutata esclusivamente tramite il simulatore esterno `mobd` con l'opzione `-b`. Il simulatore è deterministico e richiede in input il file `x.txt` (2000 valori reali, un valore per riga).

2 Fondamento teorico dell'algoritmo

2.1 Ottimizzazione non vincolata e condizioni di ottimalità

Il problema è un'ottimizzazione *non vincolata* in $\mathbb{R}^n$ con $n = 2000$:

$$\min_{x \in \mathbb{R}^n} f(x).$$

Condizioni necessarie del primo ordine

Se x^* è un minimo locale di f e f è differenziabile in x^* , allora:

$$\nabla f(x^*) = 0.$$

Un punto che soddisfa questa condizione è detto *punto stazionario*.

Condizioni necessarie del secondo ordine

Se x^* è un minimo locale di f e f è due volte differenziabile in x^* , allora:

$$\nabla f(x^*) = 0 \quad \text{e} \quad \nabla^2 f(x^*) \succeq 0.$$

Condizioni sufficienti del secondo ordine

Se $\nabla f(x^*) = 0$ e $\nabla^2 f(x^*) \succ 0$, allora x^* è un minimo locale stretto.

2.2 Metodi a discesa: direzioni gradient-related

Un metodo iterativo $x_{k+1} = x_k + \alpha_k d_k$ garantisce la discesa se la direzione d_k è *gradient-related*, cioè soddisfa:

$$\nabla f(x_k)^\top d_k \leq -c_1 \|\nabla f(x_k)\|^2, \quad \|d_k\| \leq c_2 \|\nabla f(x_k)\|,$$

per costanti $c_1, c_2 > 0$. La direzione del **gradiente negativo** $d_k = -\nabla f(x_k)$ soddisfa entrambe le condizioni con $c_1 = c_2 = 1$.

2.3 Block Coordinate Descent (BCD)

Partizionamento in blocchi

Si partiziona il vettore $x \in \mathbb{R}^n$ in m blocchi:

$$x = (x^1, x^2, \dots, x^m), \quad x^i \in \mathbb{R}^{n_i}, \quad \sum_{i=1}^m n_i = n.$$

Nel nostro caso: $m = 1000$ blocchi, $n_i = 2$ per ogni i (le coordinate 2D del modulo i), $n = 2000$.

BCD Gauss-Seidel (variante ciclica)

Ad ogni iterazione, si aggiorna un solo blocco i alla volta, tenendo fissi tutti gli altri. L'aggiornamento del blocco i all'epoca k è:

$$x_{k+1}^i = x_k^i - \alpha_k \nabla_{x^i} f(x_k),$$

dove $\nabla_{x^i} f$ è il *gradiente parziale* rispetto al blocco i . Nella variante **Gauss-Seidel**, i blocchi già aggiornati nell'epoca corrente vengono usati immediatamente per il calcolo dei gradienti successivi (in contrast con la variante Jacobi che usa sempre le posizioni all'inizio dell'epoca).

2.4 Randomized BCD (RBCD) — Gauss-Seidel con permutazione casuale

RBCD Gauss-Seidel

Ad ogni epoca k :

1. Si genera una permutazione casuale σ_k di $\{1, \dots, m\}$.
2. Si aggiornano i blocchi nell'ordine $\sigma_k(1), \sigma_k(2), \dots, \sigma_k(m)$.
3. Ogni aggiornamento usa le posizioni **più recenti** disponibili.

L'uso di una permutazione casuale (invece di un ordine fisso) migliora le proprietà di convergenza e riduce il rischio di comportamenti patologici legati alla struttura del problema.

2.5 Stepsize decrescente (Diminishing Stepsize)

Schema di stepsize decrescente

Lo stepsize al passo k è definito come:

$$\alpha_k = \frac{\alpha_0}{(1 + \beta \cdot k)^\gamma},$$

dove $\alpha_0 > 0$ è lo stepsize iniziale, $\beta > 0$ controlla la velocità di decrescita, e $\gamma \in (0, 1]$ è l'esponente di decrescita.

Condizioni sufficienti per la convergenza (Teorema di convergenza)

Una sequenza di stepsize $\{\alpha_k\}$ garantisce la convergenza se:

$$\lim_{k \rightarrow \infty} \alpha_k = 0 \quad \text{e} \quad \sum_{k=0}^{\infty} \alpha_k = +\infty.$$

Per il nostro schema $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$:

- $\lim_{k \rightarrow \infty} \alpha_k = 0$ è verificata per ogni $\gamma > 0$.
- $\sum_k \alpha_k = +\infty$ è verificata se e solo se $\gamma \leq 1$.

Con $\gamma = 0.6 \leq 1$, entrambe le condizioni sono soddisfatte.

Teorema di convergenza RBCD — Gauss-Seidel (da slide del corso)

Sia $f : \mathbb{R}^n \rightarrow \mathbb{R}$ continuamente differenziabile con gradiente Lipschitz. Sia $\{x_k\}$ la sequenza generata dall'algoritmo RBCD Gauss-Seidel con stepsize decrescente che soddisfa $\alpha_k \rightarrow 0$ e $\sum_k \alpha_k = +\infty$. Allora ogni punto di accumulazione della sequenza $\{x_k\}$ è un **punto stazionario** di f , cioè soddisfa $\nabla f(x^*) = 0$.

Osservazione 2.1. Nel caso non convesso, la complessità per ottenere $\|\nabla f(x_k)\| \leq \varepsilon$ è $O(\varepsilon^{-2})$ iterazioni, analoga a quella del metodo del gradiente classico.

2.6 Differenze finite in avanti per il gradiente di f_4

Poiché f_4 è una black-box senza espressione analitica, si approssima il suo gradiente parziale tramite **differenze finite in avanti** (forward finite differences):

$$\frac{\partial f_4}{\partial x_j^i}(x) \approx \frac{f_4(x + H \cdot e_{2i+j}) - f_4(x)}{H}, \quad j \in \{0, 1\},$$

dove $H > 0$ è il passo di perturbazione (nel codice $H = 10^{-4}$) ed e_k è il k -esimo vettore della base canonica di $\mathbb{R}^{2000}$.

Per $m = 1000$ moduli con 2 coordinate ciascuno, occorrono $2m = 2000$ **chiamate al simulatore** per epoca (più 1 per il valore base $f_4(x)$).

Scelta di H

Un valore di H troppo grande introduce errore di troncamento, uno troppo piccolo causa cancellazione numerica. Il valore $H = 10^{-4}$ rappresenta un buon compromesso per le scale del problema (coordinate nell'ordine di 10^1 – 10^2 km).

3 Limiti iniziali e soluzioni adottate

3.1 Problema 1: costo computazionale di f_4

Limite

Ogni valutazione del simulatore richiede circa 5 secondi. Con 2000 chiamate per epoca in modo sequenziale: $2000 \times 5 \text{ s} \approx 2.78 \text{ ore}$ per epoca. Con 20 epoche: $\approx 55.5 \text{ ore}$. Assolutamente inattuabile.

Soluzione: parallelizzazione con `ThreadPoolExecutor`.

Le 2000 perturbazioni sono indipendenti (ogni perturbazione riguarda un diverso indice del vettore x , tenendo fissi tutti gli altri). Si possono quindi eseguire *in parallelo* su più thread. Con N workers, il tempo si riduce a $\lceil 2000/N \rceil \times 5 \text{ s}$. Su una macchina con 32 CPU: $\lceil 2000/32 \rceil \times 5 \text{ s} = 63 \times 5 = 315 \text{ s} \approx 5.25 \text{ min/epoca}$.

3.2 Problema 2: coerenza della base per le differenze finite

Limite

Nella variante Gauss-Seidel, i blocchi vengono aggiornati in sequenza durante un'epoca. Se si calcolasse il gradiente di f_4 *on-the-fly* (mentre x cambia), il valore base $f_4(x)$ usato nelle differenze finite sarebbe inconsistente con il valore di x in quel momento.

Soluzione: strategia “stale-base”.

All'inizio di ogni epoca si scatta uno *snapshot* $\hat{x}$ dello stato corrente e si calcola $\hat{f}_4 = f_4(\hat{x})$. Tutte le 2000 perturbazioni usano la stessa base $\hat{x}$, garantendo che il denominatore delle differenze finite sia sempre il medesimo punto di partenza. Questo introduce una leggera staleness del gradiente di f_4 rispetto allo stato x a fine epoca, ma è accettabile poiché lo stepsize è piccolo.

3.3 Problema 3: inizializzazione e trappole locali

Limite

Un'inizializzazione casuale (ad esempio $x = 0$ o posizioni uniformi) porta tipicamente a valori elevati di f_3 , il che crea un gradiente dominante che può portare l'algoritmo lontano da buone soluzioni per i componenti f_1, f_2, f_4 .

Soluzione: warm start con $f_3 = 0$ per costruzione.

Si costruisce un'inizializzazione in cui ogni modulo è posizionato esattamente a distanza $r = 100$ dal suo punto di riferimento s^i , in modo che $f_3 = 0$ al punto iniziale.

La costruzione è:

$$x_0^i = s^i + r \cdot \frac{-s^i}{\|s^i\|},$$

cioè si parte da s^i e ci si sposta di r nella direzione opposta a s^i (verso l'origine). Si verifica che $\|x_0^i - s^i\| = r$ quindi $\phi_i = 0$ e $f_3 = 0$.

3.4 Problema 4: I/O intensivo su disco

Limite

Ogni chiamata al simulatore richiede di scrivere un file da 2000 righe su disco. Con 2000+ scritture per epoca, l'I/O su disco potrebbe diventare collo di bottiglia.

Soluzione: RAM disk (`/dev/shm`).

Tutti i file temporanei vengono scritti su `/dev/shm`, che è un filesystem in memoria RAM su Linux. Le scritture sono ordini di grandezza più veloci rispetto al disco fisico.

3.5 Problema 5: gradiente di f_1 di complessità $O(m^2)$

Limite

Il gradiente di f_1 richiede la somma su tutti gli $m - 1 = 999$ moduli per ogni blocco i . Con $m = 1000$ blocchi per epoca, la complessità totale è $O(m^2)$ per f_1 .

Soluzione: calcolo vettorizzato con NumPy.

Usando operazioni NumPy (broadcasting e `einsum`), il calcolo è eseguito in C sotto il cofano, evitando cicli Python espliciti. Questo riduce drasticamente i tempi pur mantenendo complessità $O(m^2)$.

4 Struttura del codice: panoramica

Il file principale è `mobd_optimizer.py` (393 righe). La struttura logica è la seguente:

1. **Costanti del problema** (righe 45–61): m , r , H , stazioni fisse.
2. **Interfaccia simulatore** (righe 64–125): scrittura file, chiamata `mobd`, calcolo parallelo delle 2000 perturbazioni.
3. **Funzioni di costo analitiche** (righe 128–153): f_1 , f_2 , f_3 .
4. **Gradienti parziali analitici** (righe 156–179): $\nabla_{x^i} f_1$, $\nabla_{x^i} f_2$, $\nabla_{x^i} f_3$.

5. **Warm start** (righe 182–201): inizializzazione con $f_3 = 0$.
6. **Main loop RBCD** (righe 204–312): algoritmo principale.
7. **CLI e argomenti** (righe 315–392): parsing argomenti, modalità fast/full/both.

5 Analisi riga per riga del codice

5.1 Import e costanti globali

```

1 #!/usr/bin/env python3
2 """
3 Mars Operations: Base Deployment (MOBD) - Optimizer
4 Randomized Block Coordinate Descent (RBCD), Gauss-Seidel
   variant.
5 ...
6 """

```

Listing 1: Righe 1–61 — Header, import e costanti

Riga 1: Shebang Unix — permette di eseguire lo script direttamente da terminale come `./mobd_optimizer.py` senza invocare esplicitamente Python.

Righe 2–35: Docstring del modulo. Descrive il problema, l'algoritmo, la strategia di parallelizzazione e il checkpointing. Questa è la documentazione pubblica del modulo, utile per capire il progetto senza leggere il codice.

```

1 import numpy as np
2 import subprocess
3 import os
4 import sys
5 import time
6 import argparse
7 from concurrent.futures import ThreadPoolExecutor,
   as_completed

```

Listing 2: Righe 37–43 — Import delle librerie

- **numpy**: algebra lineare vettorizzata (array, exp, einsum, ecc.).
- **subprocess**: per invocare il simulatore `mobd` come processo esterno.
- **os**: per operazioni sul filesystem (path, cpu_count).
- **sys**: accesso agli argomenti della riga di comando (usato implicitamente).
- **time**: misurazione del tempo di esecuzione per ogni epoca.
- **argparse**: parsing degli argomenti CLI (`-mode`, `-alpha0`, ecc.).
- **ThreadPoolExecutor**, **as_completed**: pool di thread per eseguire le 2000 perturbazioni di f_4 in parallelo.

```

1 M      = 1000
2 R      = 100.0
3 H      = 1e-4          # finite-difference step per  $df_4/dx$ 
4 LOG1R2 = np.log(1.0 + R * R)
5
6 STATIONS = np.array([[0., 0.], [100., 100.], [-100., 100.],
7                      [-100., -100.], [100., -100.]], dtype=
                        np.float64)
8
9 _DIR      = os.path.dirname(os.path.abspath(__file__))
10 SIMULATOR = os.path.join(_DIR, "linux", "mobd")
11 OUTPUT    = os.path.join(_DIR, "x.txt")
12 _SHM      = "/dev/shm"
13
14 N_WORKERS = max(1, os.cpu_count() or 4)

```

Listing 3: Righe 45–61 — Costanti del problema

- $M = 1000$: numero di moduli operativi.
- $R = 100.0$: distanza operativa ideale (in km). È il valore r della traccia.
- $H = 1e-4$: passo per le differenze finite di f_4 . Valore scelto come compromesso tra errore di troncamento e cancellazione numerica.
- $\text{LOG1R2} = \log(1 + 100^2) = \log(10001)$: costante calcolata una volta sola (invece di ricalcolarla m volte), usata in f_3 e nel suo gradiente.
- **STATIONS**: array (5×2) delle stazioni fisse. **Nota chiave**: la somma vettoriale delle stazioni è $(0, 0)$: $z^0 + z^1 + z^2 + z^3 + z^4 = (0, 0)$. Questo semplifica il calcolo di $\nabla_{x^i} f_2$ come mostrato più avanti.
- **_DIR**: directory contenente lo script, usata per costruire percorsi assoluti.
- **SIMULATOR**: percorso del binario del simulatore (dipende dal SO; qui Linux).
- **OUTPUT**: percorso del file di output `x.txt`.
- **_SHM** = `"/dev/shm"`: RAM disk di Linux. File scritti qui risiedono in RAM.
- **N_WORKERS**: numero di thread paralleli = numero di CPU logiche della macchina. Il `max(1, ...)` garantisce almeno 1 worker anche su macchine senza `cpu_count`.

5.2 Interfaccia con il simulatore

```

1 def _write_x(x_flat: np.ndarray, path: str) -> None:
2     """Write 2000 floats (one per line) to a RAM-disk path.
3     """
4     np.savetxt(path, x_flat, fmt="%15g")
5
6 def eval_f4(x_flat: np.ndarray, path: str) -> float:

```

```

7      """Invoke mobd <path> -b and return the float value
      of f4."""
8      _write_x(x_flat, path)
9      proc = subprocess.run([SIMULATOR, path, "-b"],
10                           capture_output=True, text=True,
11                           check=True)
12      return float(proc.stdout.strip())

```

Listing 4: Righe 68–78 — Scrittura file e chiamata al simulatore

_write_x:

- Riceve `x_flat`: array monodimensionale di 2000 float (formato del simulatore).
- `np.savetxt(path, x_flat, fmt="%.15g")`: scrive un valore per riga con 15 cifre significative (%g evita notazione scientifica non necessaria).

eval_f4:

- Scrive il vettore su file, poi lancia il simulatore con `subprocess.run`.
- `[SIMULATOR, path, "-b"]`: lista di argomenti (evita shell injection).
- `capture_output=True`: cattura stdout/stderr senza stamparli.
- `text=True`: decodifica l'output come stringa UTF-8.
- `check=True`: solleva `CalledProcessError` se il simulatore termina con codice diverso da 0.
- `float(proc.stdout.strip())`: converte la stringa di output nel valore numerico.

```

1 def _perturb_eval(args: tuple) -> float:
2     """Worker: perturb x_flat at index idx by +H and eval
      f4."""
3     x_snap, idx, worker_path = args
4     p = x_snap.copy()
5     p[idx] += H
6     return eval_f4(p, worker_path)
7
8
9 def eval_f4_allpartials(
10     x_snap: np.ndarray,
11     f4_base: float,
12 ) -> np.ndarray:
13     """
14     Compute f4 partial-gradient for every block
      simultaneously.
15     Uses forward FD: df4/dx^i_j ~= (f4(x+H*e_{2i+j}) -
      f4_base) / H
16     Returns g_f4 : (m, 2) array.

```

```

17     """
18     worker_paths = [os.path.join(_SHM, f"mobd_w{k}.txt")
19                       for k in range(N_WORKERS)]
20     tasks = []
21     for i in range(M):
22         tasks.append((x_snap, 2 * i, worker_paths[(2 *
23             i) % N_WORKERS]))
24         tasks.append((x_snap, 2 * i + 1, worker_paths[(2 *
25             i + 1) % N_WORKERS]))
26
27     results = [None] * (2 * M)
28     with ThreadPoolExecutor(max_workers=N_WORKERS) as pool:
29         future_to_idx = {
30             pool.submit(_perturb_eval, t): k
31             for k, t in enumerate(tasks)
32         }
33         for fut in as_completed(future_to_idx):
34             results[future_to_idx[fut]] = fut.result()
35
36     g_f4 = np.empty((M, 2), dtype=np.float64)
37     for i in range(M):
38         g_f4[i, 0] = (results[2 * i] - f4_base) / H
39         g_f4[i, 1] = (results[2 * i + 1] - f4_base) / H
40     return g_f4

```

Listing 5: Righe 81–125 — Worker function e calcolo parallelo del gradiente di f_4

`_perturb_eval:`

- Funzione worker eseguita su ogni thread.
- `x_snap.copy()`: copia difensiva per evitare race condition (ogni thread lavora su una copia indipendente del vettore).
- `p[idx] += H`: aggiunge il passo H alla coordinata `idx`.
- Chiama `eval_f4(p, worker_path)` e restituisce il valore scalare.

`eval_f4_all_partials:`

- `worker_paths`: N path distinti su `/dev/shm`, uno per thread. I thread non si sovrascrivono a vicenda perché ognuno scrive sul proprio file.
- **Costruzione dei task**: per ogni modulo i si creano 2 task (uno per coordinata). La tupla contiene `(x_snap, indice, path_worker)`. L'assegnazione `(2*i) % N_WORKERS` distribuisce i task ciclicamente tra i worker (round-robin), bilanciando il carico.
- `results = [None] * (2*M)`: lista pre-allocata di 2000 slot.
- **Pool di thread**: `ThreadPoolExecutor` mantiene N thread attivi. `pool.submit(_perturb_eval, t)` schedula il task t sul pool e restituisce un `Future`.

- `future_to_idx`: dizionario che mappa ogni Future al suo indice originale, necessario per ricostruire l'ordine dei risultati.
- `as_completed(...)`: itera sui Future man mano che completano (non in ordine), scrivendo il risultato nella posizione corretta di `results`.
- **Costruzione di `g_f4`**: applica la formula delle differenze finite $\hat{\partial}_{ij}f_4 = (\text{results}[2i + j] - f_4^{\text{base}})/H$.
- Restituisce un array $(m \times 2)$ del gradiente approssimato di f_4 .

Osservazione 5.1. Si usa `ThreadPoolExecutor` (thread, non processi) perché il collo di bottiglia è I/O (scrittura file + attesa del simulatore esterno), non CPU. Il GIL (Global Interpreter Lock) di Python non è un problema qui: i thread si bloccano sull'I/O e il simulatore gira come processo separato.

5.3 Funzioni di costo analitiche

```

1 def compute_f1(X: np.ndarray) -> float:
2     """f1 = 0.1 * sum_{i<j} (1 - exp(-||x^i - x^j||^2))"""
3     diff = X[:, None, :] - X[None, :, :]          # (m, m,
4         2)
5     sq = np.einsum("ijk,ijk->ij", diff, diff)      # (m, m)
6     return float(0.1 * (M * (M - 1) / 2 - np.triu(np.exp(-
7         sq), k=1).sum()))
8
9 def compute_f2(X: np.ndarray) -> float:
10    """f2 = (1/m) * sum_i sum_q ||x^i - z^q||^2"""
11    diff = X[:, None, :] - STATIONS[None, :, :]    # (m, 5,
12        2)
13    return float(np.einsum("ijk,ijk->", diff, diff) / M)
14
15 def compute_f3(X: np.ndarray, S: np.ndarray) -> float:
16    """f3 = 1000 * sum_i (log(1+||x^i-s^i||^2)/log(1+r^2) -
17        1)^2"""
18    d2 = np.einsum("ij,ij->i", X - S, X - S)
19    phi = np.log1p(d2) / LOG1R2 - 1.0
20    return float(1000.0 * phi @ phi)
21
22 def compute_f_known(X: np.ndarray, S: np.ndarray) -> float:
23     return compute_f1(X) + compute_f2(X) + compute_f3(X, S)

```

Listing 6: Righe 132–153 — f_1 , f_2 , f_3 e loro somma

`compute_f1:`

- `X[:, None, :]` - `X[None, :, :]`: broadcasting NumPy — crea la matrice $(m \times m \times 2)$ di tutte le differenze $x^i - x^j$.
- `np.einsum("ijk,ijk->ij", diff, diff)`: calcola $\|x^i - x^j\|^2$ per ogni coppia $(i, j) \rightarrow$ matrice $(m \times m)$.
- $f_1 = 0.1 \sum_{i < j} (1 - e^{-\|x^i - x^j\|^2}) = 0.1 \left[\binom{m}{2} - \sum_{i < j} e^{-\|x^i - x^j\|^2} \right]$.
- `np.triu(..., k=1)`: triangolare superiore con $k = 1$ esclude la diagonale, sommando solo le coppie $i < j$.

compute_f2:

- `X[:, None, :]` - `STATIONS[None, :, :]`: differenze $(m \times 5 \times 2)$.
- `np.einsum("ijk,ijk->", ...)`: somma di tutti i quadrati scalari (contrazione completa).
- Risultato diviso per m .

compute_f3:

- `X` - `S`: differenza componente per componente $(m \times 2)$.
- `np.einsum("ij,ij->i", ...)`: norma quadratica per riga $\rightarrow$ vettore m di $\|x^i - s^i\|^2$.
- `np.log1p(d2)`: $\log(1 + d^2)$ — `log1p` è numericamente più stabile di `log(1+x)` per x piccoli.
- `phi`: vettore m di $\phi_i = \log(1 + \|x^i - s^i\|^2) / \log(1 + r^2) - 1$.
- `phi @ phi`: prodotto scalare $\sum_i \phi_i^2$ — scrittura compatta per la somma dei quadrati.

5.4 Gradienti parziali analitici per blocco

```

1 def grad_f1_i(i: int, X: np.ndarray) -> np.ndarray:
2     """df1/dx^i = 0.2 * sum_{j!=i} (x^i-x^j) * exp(-||x^i-x
      ^j||^2)"""
3     d = X[i] - X
4     sq = np.einsum("ij,ij->i", d, d)
5     w = np.exp(-sq)
6     w[i] = 0.0
7     return 0.2 * (w[:, None] * d).sum(axis=0)
8
9
10 def grad_f2_i(i: int, X: np.ndarray) -> np.ndarray:
11     """df2/dx^i = (10/m)*x^i (uses sum_q z^q = 0)"""
12     return (10.0 / M) * X[i]
13
14
15 def grad_f3_i(i: int, X: np.ndarray, S: np.ndarray) -> np.
    ndarray:

```

```

16      """df3/dx^i = 4000*phi_i*(x^i-s^i) / ((1+D_i)*log(1+r
      ^2))"""
17      d = X[i] - S[i]
18      D = float(d @ d)
19      phi = np.log1p(D) / LOG1R2 - 1.0
20      return (4000.0 * phi / ((1.0 + D) * LOG1R2)) * d

```

Listing 7: Righe 160–179 — Gradienti parziali di f_1 , f_2 , f_3 per il blocco i **grad_f1_i:**

Si ricava derivando f_1 rispetto a x^i :

$$\frac{\partial f_1}{\partial x^i} = 0.1 \sum_{j \neq i} \frac{\partial}{\partial x^i} (1 - e^{-\|x^i - x^j\|^2}) = 0.2 \sum_{j \neq i} (x^i - x^j) e^{-\|x^i - x^j\|^2}.$$

- $d = X[i] - X$: array $(m \times 2)$ di tutte le differenze $x^i - x^j$ (inclusa $j = i$, che però vale $(0, 0)$).
- sq : vettore m di $\|x^i - x^j\|^2$.
- $w = \exp(-sq)$: pesi.
- $w[i] = 0.0$: annulla il contributo del termine $j = i$ (evita di sottrarlo esplicitamente).
- $(w[:, None] * d).sum(axis=0)$: somma pesata delle differenze $\rightarrow$ vettore 2D.

grad_f2_i:

Si ricava derivando f_2 rispetto a x^i :

$$\frac{\partial f_2}{\partial x^i} = \frac{2}{m} \sum_{q=0}^4 (x^i - z^q) = \frac{2}{m} \left(5x^i - \sum_{q=0}^4 z^q \right) = \frac{10}{m} x^i,$$

dove si usa il fatto che $\sum_{q=0}^4 z^q = (0, 0)$ (le 5 stazioni si bilanciano perfettamente). Questa è una **semplificazione chiave**: il gradiente è semplicemente proporzionale a x^i .

grad_f3_i:

Si ricava derivando f_3 rispetto a x^i :

$$\frac{\partial f_3}{\partial x^i} = 2 \cdot 1000 \cdot \phi_i \cdot \frac{2(x^i - s^i)}{(1 + \|x^i - s^i\|^2) \log(1 + r^2)} = \frac{4000 \phi_i (x^i - s^i)}{(1 + D_i) \log(1 + r^2)},$$

dove $D_i = \|x^i - s^i\|^2$ e $\phi_i = \log(1 + D_i) / \log(1 + r^2) - 1$.

- $d = X[i] - S[i]$: vettore $(x^i - s^i) \in \mathbb{R}^2$.
- $D = d @ d$: $\|x^i - s^i\|^2$ come scalare.
- phi : ϕ_i calcolato per il modulo i .

- Il risultato è un vettore 2D scalato per il coefficiente $4000\phi_i/((1+D_i)\log(1+r^2))$.

5.5 Warm start

```

1 def warm_start() -> tuple:
2     """
3     Ref points:  $s^i = ((-1)^{i \cdot 0.2}, (-1)^{i \cdot 0.2})$ ,  $i=1..m$ .
4     Place each module at distance  $r$  from  $s^i$  toward origin:
5          $x^i = s^i + r * (-s^i / ||s^i||)$ 
6      $||x^i - s^i|| = r \Rightarrow \phi_i = 0 \Rightarrow f_3 = 0$ .
7     """
8     idx = np.arange(1, M + 1, dtype=np.float64)
9     sign = np.where(idx % 2 == 0, 1.0, -1.0)
10    S = np.column_stack([sign * idx * 0.2, sign * idx *
11                        0.2])
12    nrm = np.linalg.norm(S, axis=1, keepdims=True)
13    safe = np.where(nrm < 1e-12, 1.0, nrm)
14    X = S + R * (-S / safe)
15    X[nrm[:, 0] < 1e-12] = [R, 0.0]
16    return X, S

```

Listing 8: Righe 186–201 — Inizializzazione con $f_3 = 0$

- `idx`: array $[1, 2, \dots, 1000]$ dei numeri di modulo.
- `sign`: $(-1)^i$ — vale -1 se i dispari, $+1$ se i pari.
- `S`: array $(m \times 2)$ dei punti di riferimento $s^i = ((-1)^i \cdot i \cdot 0.2, (-1)^i \cdot i \cdot 0.2)$.
- `nrm`: norma $\|s^i\|$ per ogni modulo.
- `safe`: versione di `nrm` con i valori troppo piccoli sostituiti con 1 (guard contro divisione per zero per $s^i \approx 0$).
- `X = S + R * (-S / safe)`: formula geometrica. Si parte da s^i e ci si sposta di $r = 100$ nella direzione $-s^i/\|s^i\|$ (verso l'origine). Così $\|x^i - s^i\| = r$ esattamente, e quindi $\phi_i = 0$ e $f_3 = 0$.
- `X[nrm < 1e-12] = [R, 0.0]`: caso degenero $s^i \approx 0$ (solo per $i = 0$, che nel nostro caso non esiste perché i parte da 1). Fallback: posiziona in $(R, 0)$.
- Restituisce la coppia `(X, S)`: posizioni iniziali e punti di riferimento.

5.6 Loop principale RBCD

```

1 def rbcd(
2     n_epochs: int = 20,
3     alpha_0: float = 0.02,
4     beta: float = 0.05,
5     gamma: float = 0.60,
6     seed: int = 42,
7     use_f4_grad: bool = True,

```



```

8     resume:         bool    = True,
9 ) -> np.ndarray:

```

Listing 9: Righe 208–268 — Firma funzione e inizializzazione

- `n_epochs`: numero di epoche (sweep completi su tutti gli m blocchi).
- `alpha_0`: stepsize iniziale α_0 .
- `beta`, `gamma`: parametri dello schedule decrescente $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$.
- `seed`: seme per la generazione delle permutazioni casuali (riproducibilità).
- `use_f4_grad`: se `True` usa le differenze finite per f_4 (modalità spec-compliant); se `False` usa solo i gradienti analitici di $f_1 + f_2 + f_3$ (modalità diagnostica veloce).
- `resume`: se `True` e `x.txt` esiste, riprende da quel punto invece che dal warm start.

```

1     rng = np.random.default_rng(seed)
2
3     X, S = warm_start()
4     print("Warm start computed (f3 = 0 by construction)")
5
6     if resume and os.path.exists(OUTPUT):
7         try:
8             xl = np.loadtxt(OUTPUT)
9             if xl.size == 2 * M:
10                 X = xl.reshape(M, 2)
11                 print(f"Resumed from existing {OUTPUT}")
12         except Exception as exc:
13             print(f"Could not load {OUTPUT}: {exc}")
14
15     np.savetxt(OUTPUT, X.flatten(), fmt="%.15g")
16
17     t0      = time.time()
18     tmp0    = os.path.join(_SHM, "mobd_init.txt")
19     f4_now  = eval_f4(X.flatten(), tmp0)
20     fk      = compute_f_known(X, S)
21     total   = fk + f4_now
22     best    = total
23     best_X  = X.copy()
24     np.savetxt(OUTPUT, best_X.flatten(), fmt="%.15g")

```

Listing 10: Righe 230–256 — Inizializzazione e prima valutazione

- `np.random.default_rng(seed)`: generatore random moderno (NumPy ≥ 1.17), con `seed` fisso per reproducibilità.
- `warm_start()`: calcola posizioni iniziali con $f_3 = 0$.

- Blocco `if resume`: tenta di caricare una soluzione precedente da `x.txt`. Controlla che abbia esattamente 2000 valori. In caso di errore di lettura, usa il warm start (il `try/except` è un guard generico).
- `np.savetxt(OUTPUT, X.flatten(), ...)`: salva immediatamente il checkpoint iniziale (garantisce che `x.txt` sia sempre valido anche se il programma viene interrotto prima della prima epoca).
- `X.flatten()`: converte l'array ($m \times 2$) in un vettore monodimensionale di 2000 elementi (formato richiesto dal simulatore).
- Prima valutazione di f_4 e delle componenti analitiche: stabilisce il valore iniziale e imposta `best = total`, `best_X = X.copy()`.

```

1     for ep in range(n_epochs):
2         t_ep = time.time()
3         alpha = alpha_0 / (1.0 + beta * ep) ** gamma
4         order = rng.permutation(M)
5
6         if use_f4_grad:
7             snap = X.flatten()
8             f4_base = f4_now
9             t_fd = time.time()
10            g_f4 = eval_f4_all_partials(snap, f4_base)
11            print(f" [f4 FD done in {time.time()-t_fd:.1f}
12                  s]", flush=True)
13        else:
14            g_f4 = np.zeros((M, 2), dtype=np.float64)
15
16        for i in order:
17            g = (grad_f1_i(i, X)
18                + grad_f2_i(i, X)
19                + grad_f3_i(i, X, S)
20                + g_f4[i])
21            X[i] -= alpha * g

```

Listing 11: Righe 269–290 — Loop principale: un'epoca

- `alpha = alpha_0 / (1.0 + beta * ep) ** gamma`: stepsize decrescente $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$ valutato all'epoca $k = ep$.
- `order = rng.permutation(M)`: permutazione casuale degli indici $\{0, 1, \dots, 999\}$ — questo è il cuore della strategia **randomizzata**.
- **Stale-base**: `snap = X.flatten()` è lo snapshot all'inizio dell'epoca; `f4_base = f4_now` è il valore di f_4 calcolato alla fine dell'epoca precedente (e quindi corrispondente a `snap`). Questo garantisce la coerenza delle differenze finite.
- **Gauss-Seidel**: il ciclo `for i in order` aggiorna i blocchi sequenzialmente nell'ordine casuale. Ogni aggiornamento `X[i] -= alpha * g` modifica imme-

diatamente X , per cui i blocchi successivi vedono le posizioni già aggiornate — questo è esattamente la caratteristica della variante Gauss-Seidel.

- $g_f4[i]$: il gradiente di f_4 per il blocco i è quello calcolato all'inizio dell'epoca (stale), non viene ricalcolato durante l'epoch.
- $g = grad_f1_i + grad_f2_i + grad_f3_i + g_f4[i]$: gradiente totale del blocco i , somma dei quattro contributi.

```

1      tmp_ep = os.path.join(_SHM, "mobd_epoch.txt")
2      f4_now = eval_f4(X.flatten(), tmp_ep)
3      fk      = compute_f_known(X, S)
4      total   = fk + f4_now
5      dt      = time.time() - t_ep
6
7      print(f"Epoch {ep+1:3d}/{n_epochs}  total={total:.2f}")
8          f"    (f1={compute_f1(X):.2f}  f2={compute_f2(X):.2f})"
9          f"    f3={compute_f3(X,S):.2f}  f4={f4_now:.2f})"
10         f"    a={alpha:.5f}  t={dt:.1f}s",
11         flush=True)
12
13     if total < best:
14         best      = total
15         best_X    = X.copy()
16         np.savetxt(OUTPUT, best_X.flatten(), fmt="%.15g")
17         print(f"  New best! Saved to {OUTPUT}", flush=True)

```

Listing 12: Righe 292–311 — Fine epoca: valutazione, log e checkpointing

- $f4_now = eval_f4(X.flatten(), tmp_ep)$: una sola chiamata al simulatore per ottenere il costo f_4 aggiornato. Questo valore servirà anche come $f4_base$ per la prossima epoca.
- $total = fk + f4_now$: costo totale $f_1 + f_2 + f_3 + f_4$.
- **Checkpointing**: se il costo totale è migliorato rispetto al miglior valore precedente, si salva immediatamente la soluzione in `x.txt`. Questo garantisce che anche in caso di interruzione, si conserva sempre la miglior soluzione trovata finora.
- `flush=True`: forza la scrittura immediata su stdout, utile per monitorare il progresso in esecuzioni lunghe.

5.7 Modalità operative: fast, full, both

```

1     if args.mode in ("fast", "both"):
2         rbcd(
3             n_epochs      = args.fast_epochs,
4             alpha_0       = args.alpha0,
5             use_f4_grad   = False,      # solo gradiente
                                   analitico
6             resume       = resume,
7         )
8         resume = True
9
10    if args.mode in ("full", "both"):
11        rbcd(
12            n_epochs      = args.full_epochs,
13            alpha_0       = args.alpha0 * 0.5,  # step piu'
                                   piccolo dopo warm-up
14            use_f4_grad   = True,
15            resume       = True,
16        )

```

Listing 13: Righe 363–391 — Entry point con le tre modalità

Le tre modalità sono:

- **fast**: usa solo $\nabla(f_1 + f_2 + f_3)$ senza mai chiamare il simulatore durante le epoche. Ogni epoca costa $O(\text{ms})$ invece di 5+ minuti. Utile per scendere velocemente nella direzione di $f_1 + f_2 + f_3$. **Limite**: può peggiorare f_4 perché non ne minimizza il gradiente.
- **full**: modalità spec-compliant. Ogni epoca calcola le 2000 perturbazioni di f_4 in parallelo. È la modalità da usare per la competizione.
- **both**: prima esegue la fase **fast** (warm-up) poi la fase **full** con stepsize dimezzato ($\alpha_0 \times 0.5$) per affinare la soluzione. Nella fase 2 si usa **resume=True**, cioè si parte dalla soluzione trovata nella fase 1.

6 Convergenza: garanzie teoriche

6.1 Verifica delle condizioni per la convergenza

Convergenza del RBCD Gauss-Seidel con stepsize decrescente

Sia $f = f_1 + f_2 + f_3 + f_4$ con le seguenti ipotesi:

1. f è continuamente differenziabile su $\mathbb{R}^{2000}$.
2. ∇f è Lipschitz continuo (soddisfatto poiché f_1, f_2, f_3 hanno gradienti

Lipschitz, e si assume che anche f_4 abbia questa proprietà).

3. Lo stepsize soddisfa: $\alpha_k \rightarrow 0$ e $\sum_k \alpha_k = +\infty$.

Allora ogni punto di accumulazione della sequenza $\{X_k\}$ generata dall'algoritmo RBCD Gauss-Seidel è un **punto stazionario** di f , cioè $\nabla f(X^*) = 0$.

Verifica per il nostro schedule $\alpha_k = \alpha_0/(1 + \beta k)^\gamma$ con $\alpha_0 = 0.01$, $\beta = 0.05$, $\gamma = 0.60$:

$$\lim_{k \rightarrow \infty} \alpha_k = 0 \quad \checkmark, \quad \sum_{k=0}^{\infty} \alpha_k = \sum_{k=0}^{\infty} \frac{\alpha_0}{(1 + \beta k)^\gamma} = +\infty \quad (\gamma = 0.6 < 1) \quad \checkmark.$$

6.2 Complessità

Nel caso non convesso, per ottenere $\min_{k \leq K} \|\nabla f(X_k)\|^2 \leq \varepsilon$, il numero di iterazioni (chiamate al gradiente) richieste è:

$$K = O\left(\frac{1}{\varepsilon}\right) \cdot \frac{1}{\alpha_K} = O\left(\frac{1}{\varepsilon^2}\right),$$

analoga alla complessità del metodo del gradiente standard per problemi non convex.

6.3 Accuratezza del gradiente di f_4

L'errore delle differenze finite in avanti è:

$$\left| \frac{f_4(x + He_k) - f_4(x)}{H} - \frac{\partial f_4}{\partial x_k}(x) \right| = O(H),$$

per $H \rightarrow 0$. Con $H = 10^{-4}$, l'errore di troncamento è dell'ordine $10^{-4} \cdot L$, dove L è la costante di Lipschitz di ∇f_4 .

Osservazione 6.1. In teoria, le differenze finite introducono un errore nel gradiente che viola strettamente la condizione gradient-related. Tuttavia, per errori sufficientemente piccoli, i metodi a gradiente approssimato mantengono le stesse garanzie di convergenza a un punto stazionario approssimato.

7 Riepilogo delle scelte progettuali

8 Come eseguire il codice

```
1 # Modalità FULL (spec-compliant, raccomandata per la
   competizione):
2 python mobd_optimizer.py --mode full --full-epochs 20
3
4 # Modalità FAST (solo gradienti analitici, per debug):
```

Aspetto	Problema/Vincolo	Soluzione adottata
Algoritmo	Dimensione $n = 2000$, no struttura convessa garantita	RBCD Gauss-Seidel, convergenza a punti stazionari
Gradiente f_4	Black-box, no formula analitica	Differenze finite in avanti, $H = 10^{-4}$
Costo computazionale	2000 chiamate/epoca $\times 5s = 2.8h$ sequenziale	ThreadPoolExecutor con N worker paralleli
I/O	Scritture frequenti su disco $\rightarrow$ collo di bottiglia	RAM disk <code>/dev/shm</code>
Inizializzazione	Rischio trappole locali con $f_3 \gg 0$	Warm start costruttivo con $f_3 = 0$
Coerenza FD	Base inconsistente durante l'epoch Gauss-Seidel	Strategia stale-base: snapshot a inizio epoca
Robustezza	Interruzioni durante esecuzioni lunghe	Checkpointing: salva best_X dopo ogni epoca
Stepsize	Garantire convergenza teorica	Schedule decrescente $\alpha_k = \alpha_0/(1 + \beta k)^\gamma$, $\gamma = 0.6$

Tabella 1: Riepilogo delle principali scelte progettuali.

```

5 python mobd_optimizer.py --mode fast --fast-epochs 100
6
7 # Modalita' BOTH (warm-up analitico poi full):
8 python mobd_optimizer.py --mode both --fast-epochs 50 --
  full-epochs 20
9
10 # Riprendere da x.txt esistente (default) o ricominciare
    dal warm start:
11 python mobd_optimizer.py --mode full --no-resume
12
13 # Controllare la soluzione con il simulatore:
14 ./linux/mobd x.txt -b      # solo f4
15 ./linux/mobd x.txt -t      # tutto (solo per verifica
    manuale, non nel codice)

```

Listing 14: Comandi di esecuzione

Parametri disponibili:

- `-alpha0`: stepsize iniziale (default 0.01)
- `-beta`: parametro β dello schedule (default 0.05)
- `-gamma`: esponente γ dello schedule (default 0.60)
- `-seed`: seme random (default 42)

- `-workers`: numero di thread per f_4 (default: `os.cpu_count()`)
- `-no-resume`: ignora `x.txt` esistente, riparte dal warm start

9 Vincoli e regole della competizione rispettati

- **Nessuna libreria esterna di ottimizzazione**: tutte le routine (BCD, gradiente, FD) sono implementate da zero in Python/NumPy. Non si usa `scipy.optimize`, `cvxpy`, o librerie equivalenti.
- **Solo opzione `-b`**: il simulatore viene invocato esclusivamente con `-b` (solo f_4). L'opzione `-t` (totale) non viene mai usata nel codice.
- **Formato di output corretto**: `x.txt` viene scritto con `np.savetxt` con formato `%.15g`, producendo esattamente 2000 righe con un valore reale per riga, senza righe vuote.
- **Linguaggio a scelta**: Python 3, consentito.

10 Conclusioni

Il progetto MOBD è stato affrontato come un problema di ottimizzazione non vincolata su $\mathbb{R}^{2000}$. La struttura a blocchi naturale del problema (ogni blocco = 2D di un modulo) ha suggerito l'impiego del **Randomized Block Coordinate Descent** nella variante Gauss-Seidel, con permutazione casuale ad ogni epoca per evitare bias sistematici.

La presenza di f_4 black-box ha richiesto di approssimare il gradiente tramite **differenze finite in avanti**, con parallelizzazione su thread per rendere computazionalmente praticabile ogni epoca.

Lo stepsize decrescente $\alpha_k = \alpha_0 / (1 + \beta k)^\gamma$ garantisce le condizioni teoriche di convergenza (Teorema del corso). La strategia di warm start con $f_3 = 0$ fornisce un punto di partenza strutturalmente buono, e il checkpointing garantisce la robustezza dell'esecuzione.

L'algoritmo è stato implementato in Python puro + NumPy, nel pieno rispetto delle regole della competizione, senza uso di librerie di ottimizzazione esterne.